DREW

Lecture 5: Digital Logic

CSCI 260 Computer Systems & Architecture
Fall 2022

Andy Siyuan Feng

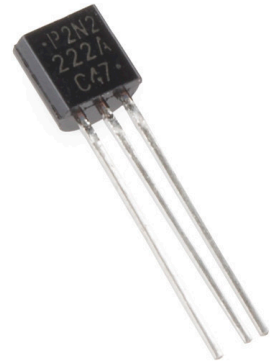
09/09/2024

Outline

- **Lecture 5**: Logical Gates and Combinational Logic: §3.2-§3.4
- **Lecture 6:** In-class Exercise on Data Rep. with attendance
- **Lecture 7**: Memory and Sequential Logic: §3.5-§3.6
  - In-class Exercise on Digital Logic with attendance

§3: Digital Logic Structures

- The lowest level component of a computer is called a *transistor*.
  - Recognizes states 0/1.
  - Can have tens of billions in a microchip nowadays.
    - **Moore's Law**

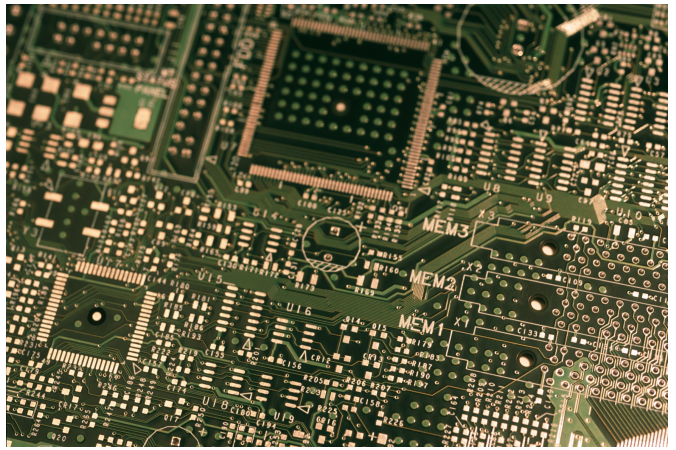


- Talked about transistors and how to get the 0s and 1s.

  Missing something here? (Materially)

  What are ADD, AND, etc. in the computer?

  Hardware, circuits

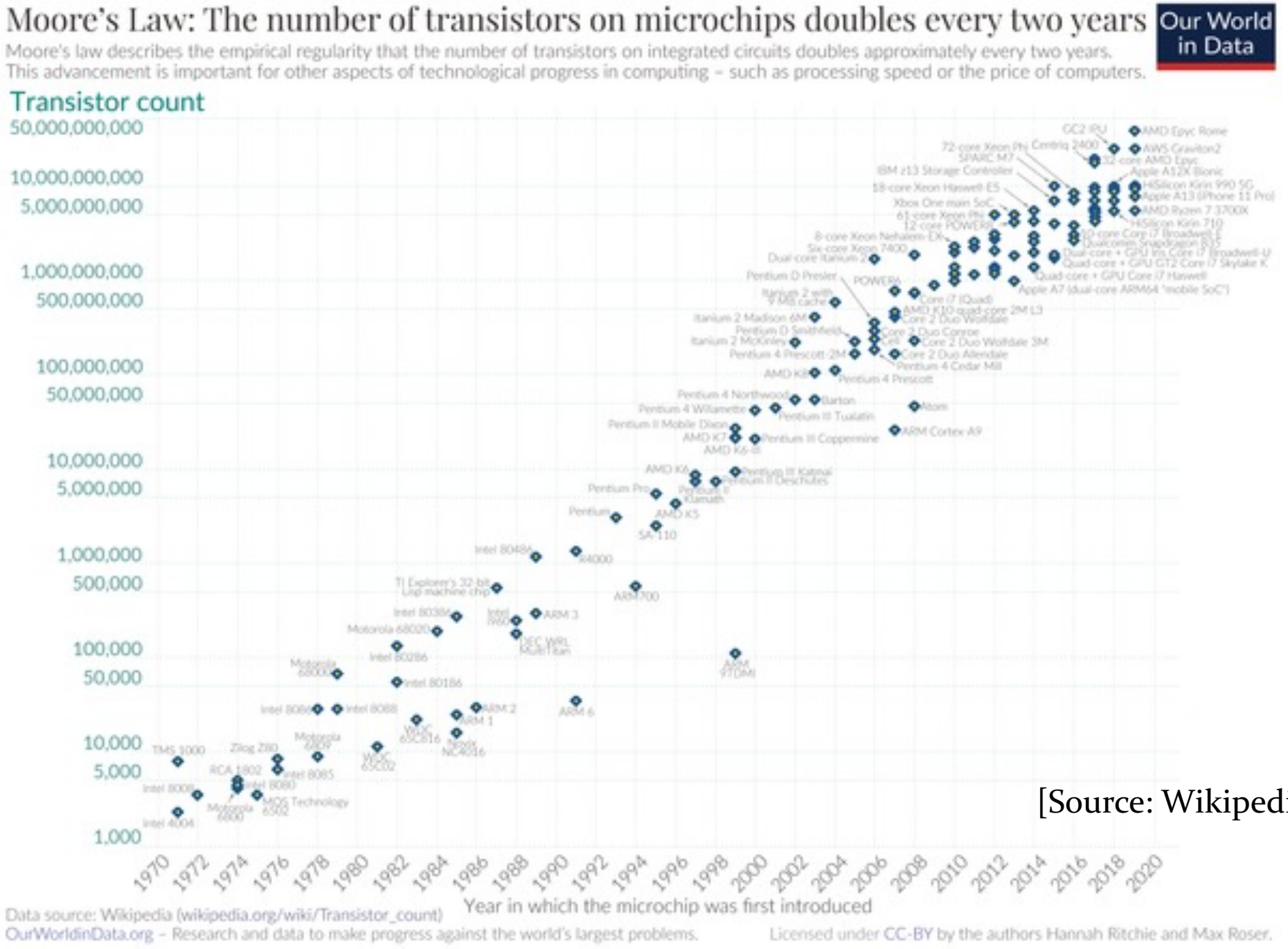


- Talked about operations with the 0s and 1s, conceptually.

DREW

Moore’s Law

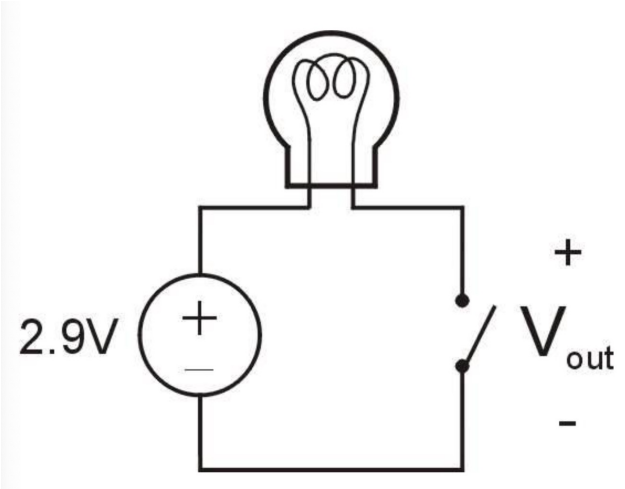
Go search:
“Nvidia Moore’s Law”



[Source: Wikipedia]

DREW

§3.1 Transistor as Switch



- Since, we are talking about hardware, circuit …
- Our transistors is the equivalence of the light switch.
  - Off → no light = 0 On = 1 → light

The Abstraction

- Now, by combining transistors, we can implement logic function

  Logical Gates: AND, OR, NOT
- Further, by combining gates, we can build *higher-level structures*

  Adder, Multiplexer, Decoder, Register, etc.
- Lastly, by combining these structures, we build *Processors*!

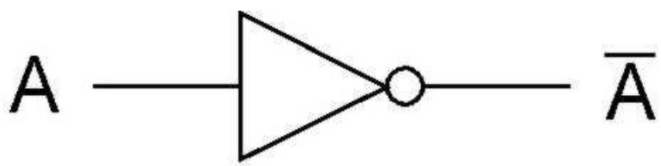
Transistors to Logical Gates

- However, the circuits from transistors to logical gates is a much more complicated topic.
  - Need to combine MOS (Metal Oxide Semiconductor) transistors of various types.
    - The p-type and n-type;
    - and their complements types (CMOS).
- More of an engineering problem, so won't go into details.

§3.2 Logical Gates

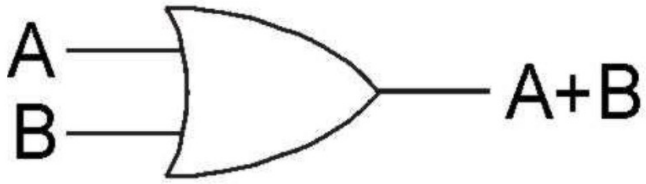
- We denote the basic logical gates as follows:

NOT

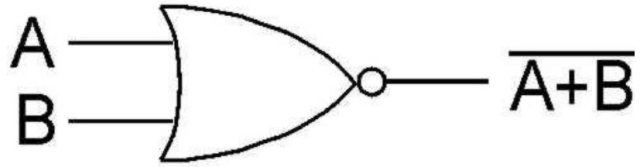


(NOT in short is just a circle before/after the gate.)

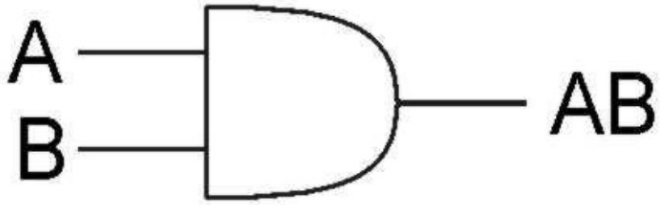
OR



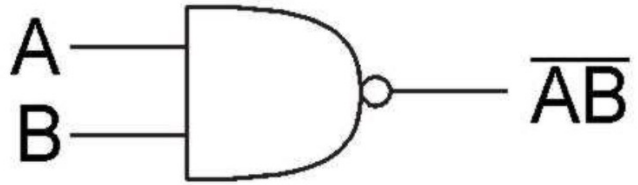
NOR



AND



NAND



Recall: we can combine logical operations/gates to achieve a new circuit.

De Morgan's Law

- **De Morgan's Law**:
  - $\text{NOT}(A + B) = \bar{A} \cdot \bar{B}$
  - ➢ $\text{NOT}(A \cdot B) = \bar{A} + \bar{B}$

(Why do these hold?)
[Essentially, we are "distributing" the NOT into the parenthesis.]

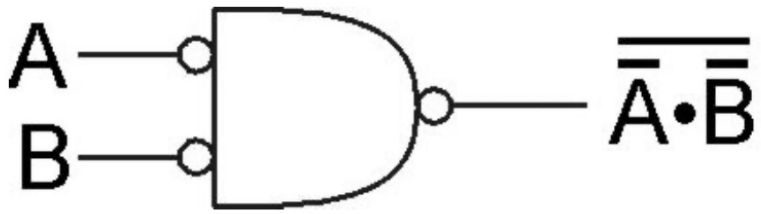
- Formally,
  prove by a Truth Table:

A	B	$A \cdot B$	$\text{NOT}(A \cdot B)$	$\bar{A}$	$\bar{B}$	$\bar{A} + \bar{B}$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

- **Q**: Given only the NOT and AND gates, can we construct an OR gate?
  Similarly, given NOT and OR, can we construct AND?

De Morgan's Law

- **De Morgan's Law:** $\text{NOT}(A + B) = \bar{A} \cdot \bar{B}$ $\text{NOT}(A \cdot B) = \bar{A} + \bar{B}$
- **Q**: Given only the NOT and AND gates, can we construct an OR gate?
- **A**: Consider this circuit,



- By De Morgan's law for AND, this becomes OR!
- To verify,

A	B	$\bar{A}$	$\bar{B}$	$\bar{A} \cdot \bar{B}$	$\overline{\bar{A} \cdot \bar{B}}$	$A + B$
0	0	1	1	1	0	0
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	0	1	1

DREW

De Morgan's Law

- **De Morgan's Law**: $\text{NOT}(A + B) = \bar{A} \cdot \bar{B}$ $\text{NOT}(A \cdot B) = \bar{A} + \bar{B}$
- **Q**: Given only the NOT and AND gates, can we construct an OR gate?
  Similarly, given NOT and OR, can we construct AND?

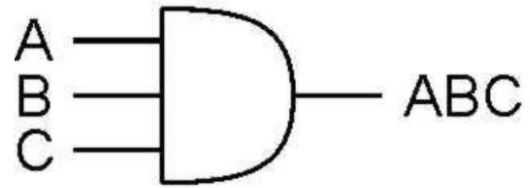
- **A**: To construct OR, we perform $\neg(\neg A \wedge \neg B)$;
  to construct AND, we perform $\neg(\neg A \vee \neg B)$.

- De Morgan's law: no need to construct truth table all the time.

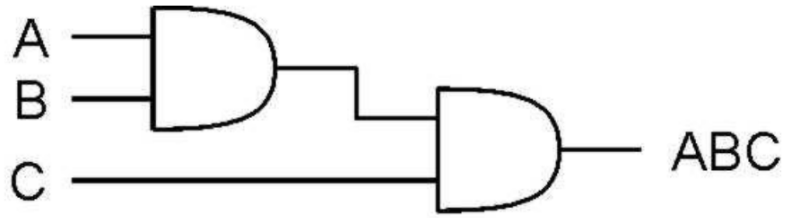
Remarks

- **Representation** of a logical function:
  1. a logical expression
  2. a truth table
  3. a logical circuit

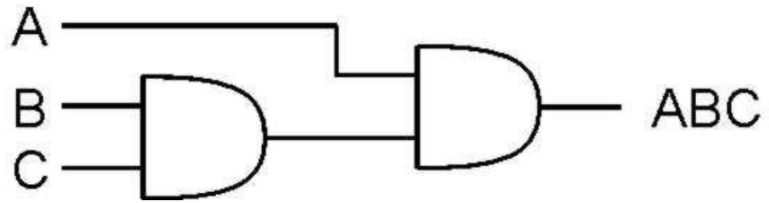
- Gates for 3 or more operands:
  (Associativity among ANDs)



is the same as,



or



One Level Up: Circuits

Quickly go over:
Decoder
Multiplexer (MUX)
Full Adder

Combinational Logic Circuit:

- Output depends only on the *current input*.
- Do not have state/"memory".
- Used for *Decision Structures*.

Sequential Logic Circuit:

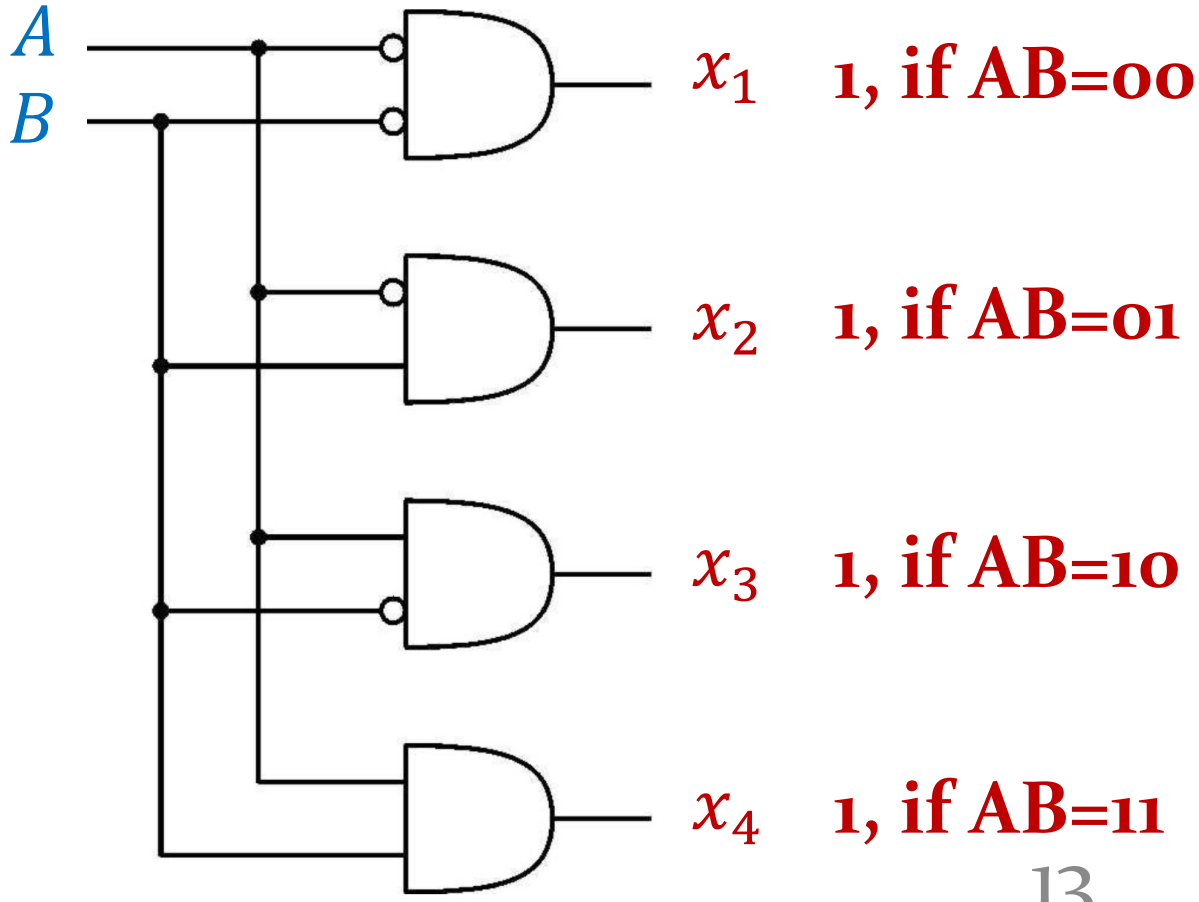
- Output depends on the *sequence of input* (past and present).
- Have states/"memory" that store (finite) info. from the past.
- Use for *Storage Structures*.

§3.3 Combinational Logic Circuit Recognizing/decoding patterns.

Decoder: For an n-bit input, there are 2^n possible output result.

But, for each input pattern, only one output is 1.

Ex.: For a 2-bit input, its decoder:



- $x_1 = \bar{A} \wedge \bar{B}$, $x_2 = \bar{A} \wedge B$, etc.
- $A = 0 \Rightarrow x_3 = x_4 = 0$:

  $B = 0 \Rightarrow x_1 = 1, x_2 = 0$;

  or $B = 1 \Rightarrow x_2 = 1, x_1 = 0$.
- Etc.

DREW

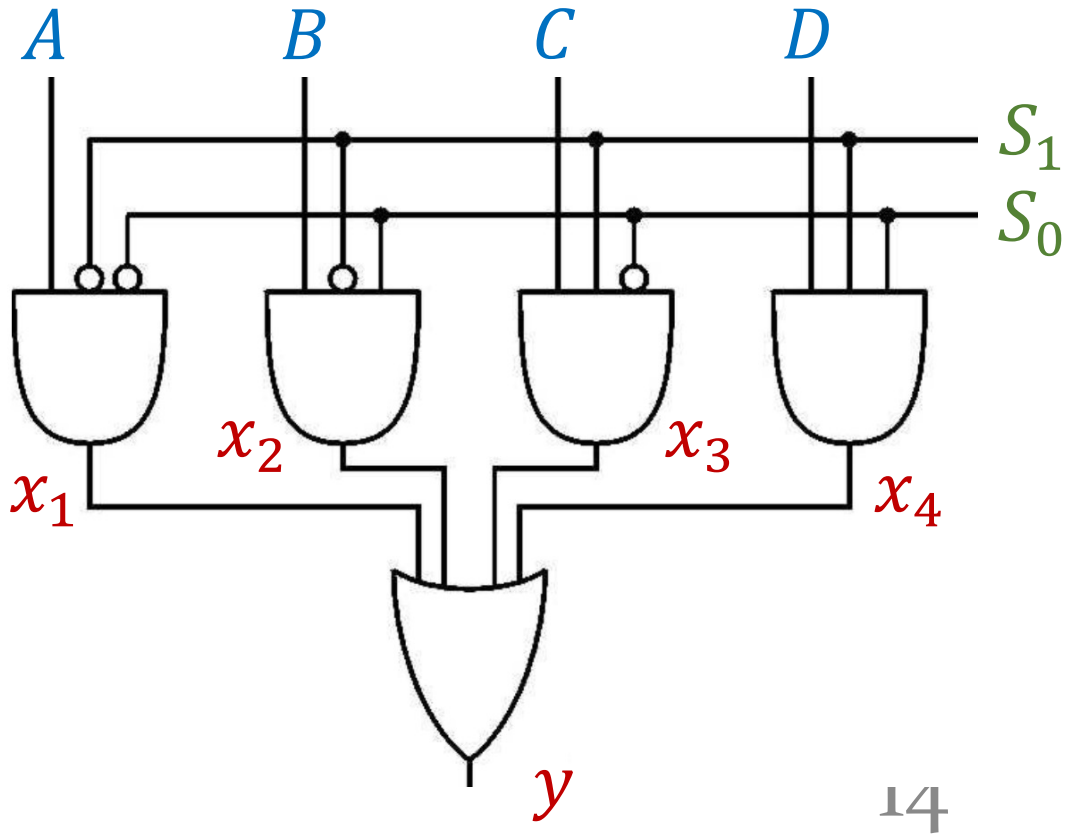
Combinational Logic Circuit Selecting the desired input.

Multiplexer (MUX): For an n-bit selector and 2^n-bit input, one output, where, based on the selector value, the output is one of the input.

Ex.: a 4-to-1 MUX with 2 selector, S_0 and S_1:

- $y = x_1 \vee x_2 \vee x_3 \vee x_4$, where
- $x_1 = A \wedge \overline{S_1} \wedge \overline{S_0}$, $x_2 = B \wedge \overline{S_1} \wedge S_0$, etc.

- $S_0 = 0 \Rightarrow x_2 = x_4 = 0$:
  $S_1 = 0 \Rightarrow x_1 = 1, x_3 = 0 \Rightarrow y = A$;
  or $S_1 = 1 \Rightarrow x_3 = 1, x_1 = 0 \Rightarrow y = C$.
- Etc.

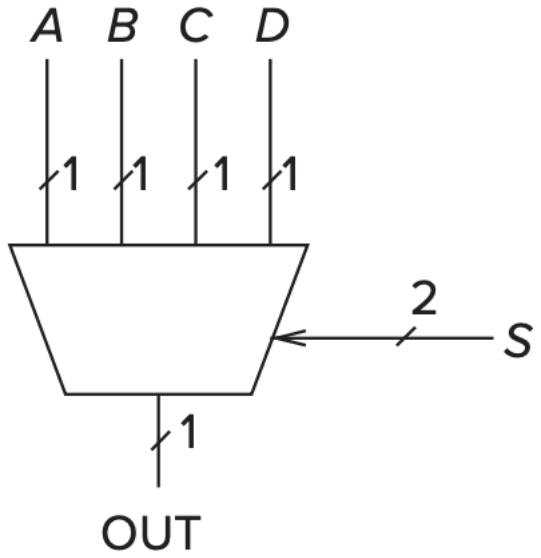


DREW

Combinational Logic Circuit

Ex.: a 4-to-1 MUX with 2 selector, S_0 and S_1:

This is the standard notation for it.



- What are we doing here? (1 word)

Abstraction

Combinational Logic Circuit

Half-Adder doesn't count the carry.

Full Adder: To add two bits and the carry-in, we utilize the *XOR* operation.

- Note that, $S_i = A_i + B_i + C_i$

$= A_i \oplus B_i \oplus C_i$ with a carry-out C_{i+1}.

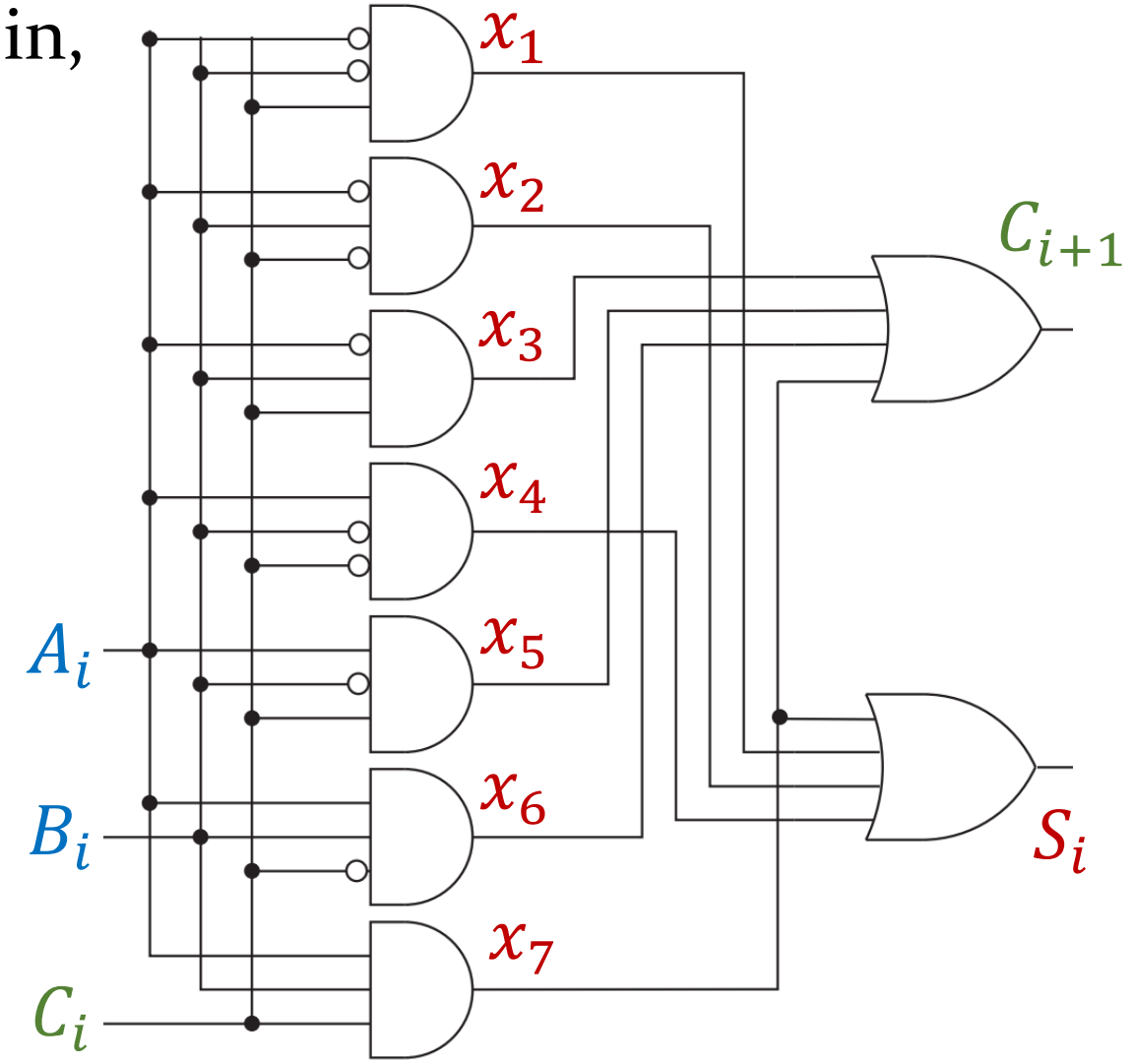
- **Ex.**: To add $A_i = B_i = C_i = 1$,

$x_1 = x_2 = x_3 = x_4 = x_5 = x_6 = 0$,

$x_7 = 1$;

$\Rightarrow S_i = x_7 \vee x_1 \vee x_2 \vee x_4 = 1$

and $C_i = x_3 \vee x_5 \vee x_6 \vee x_7 = 1$.



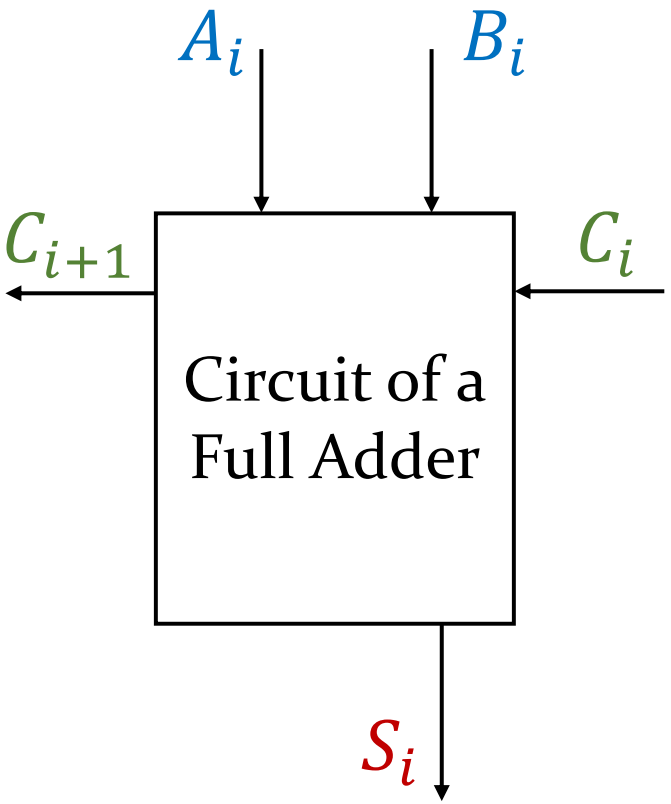
DREW

Combinational Logic Circuit

Full Adder add two bits and the carry-in, producing the sum and carry-out.

- How do we add more than 1 bit, say, 0101+1001?

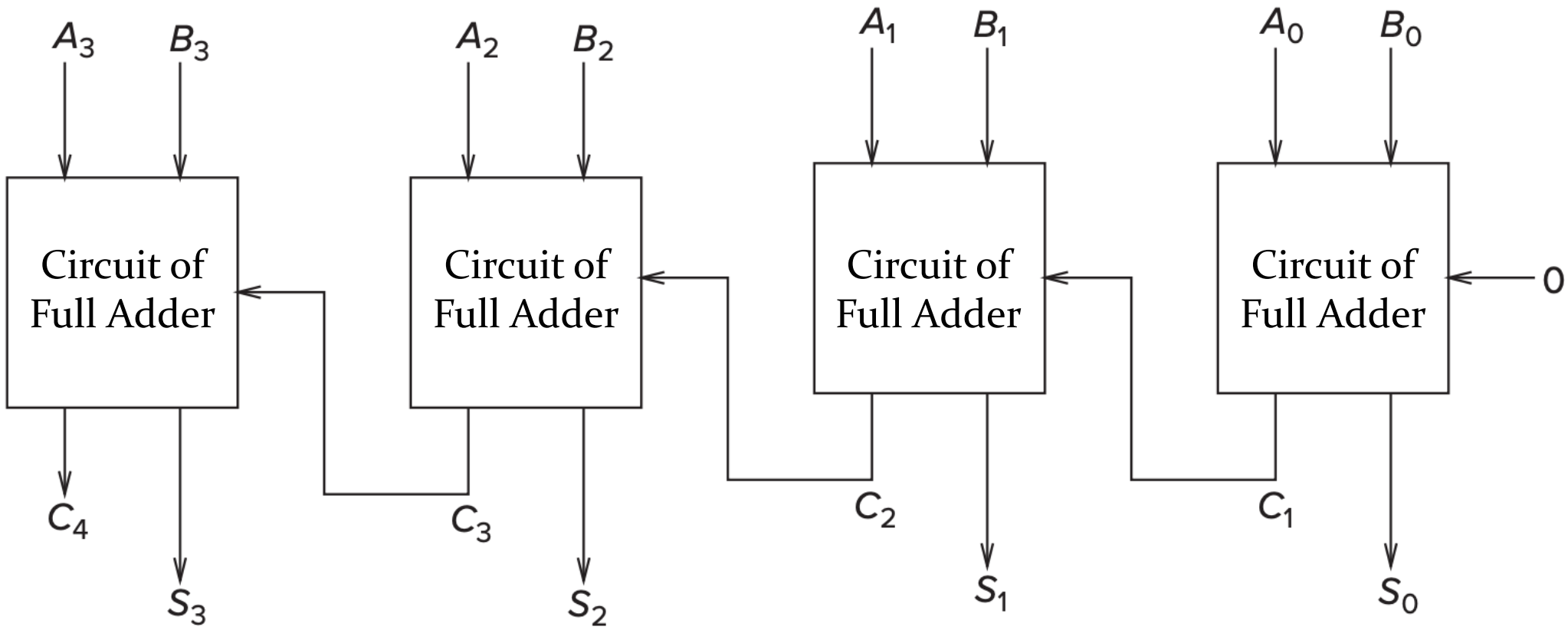
Abstraction



Combinational Logic Circuit

Full Adder add two bits and the carry-in, producing the sum and carry-out.

- How do we add more than 1 bit, say, 0101+1001?



Combinational Logic Circuit

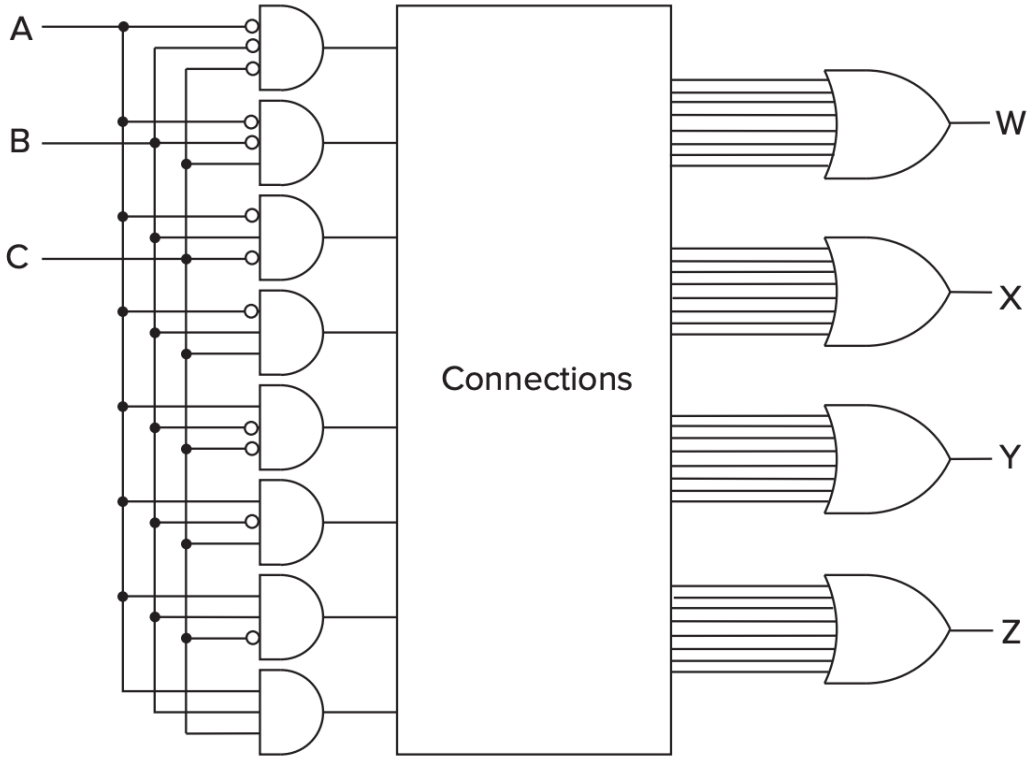
- Speaking of *abstraction*, we define a generalized building block for any circuit, the *Programmable Logic Array* (PLA).

- Given n input bits, can have 2^n different combinations of input values.
- Use (up to) 2^n AND gates to combine.

What's in between?

Up to the *program*

- Depending on the number of output bits, connect 2^n cases using OR gates.



Logical Completeness

- Turns out *PLA* can implement any logical circuit given enough gates in between.
- **Def.:** The set of logical gates {AND, OR, NOT} is *Logically Complete* because we can implement any circuit using this set of operations.
  - Using the PLA!
- Turns out, given a set of logical gates, if we can implement another set that's logically complete, e.g. {AND, OR, NOT}, then so is given set.

One Level Up: Circuits

Combinational Logic Circuit:

- Decoder
- Multiplexer (MUX)
- Full Adder

Let's talk about storage first.

- R-S Latch
- Gated D-Latch
- Registers

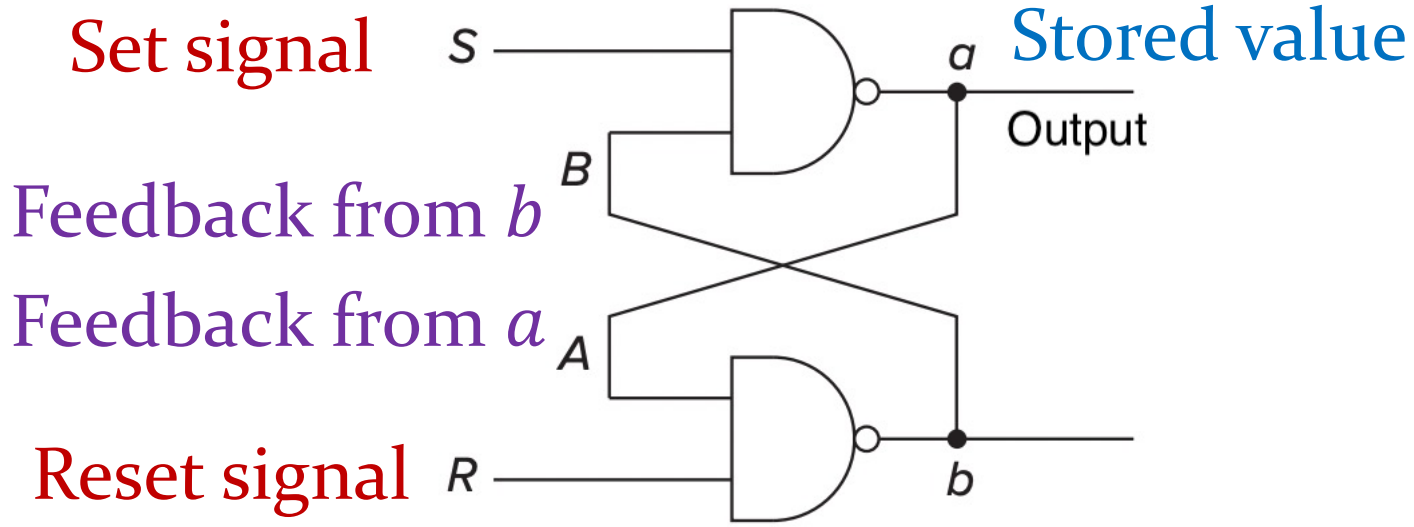
Sequential Logic Circuit:

- Use for *Storage Structures*.

§3.4 Basic Storage Elements

The R-S Latch: Is a structure that can store a single bit value within the latch, the output.

- By utilizing *feedback*, an R-S latch can either *Reset* and store a 0 to the latch, or *Set* and store a 1 to the latch.

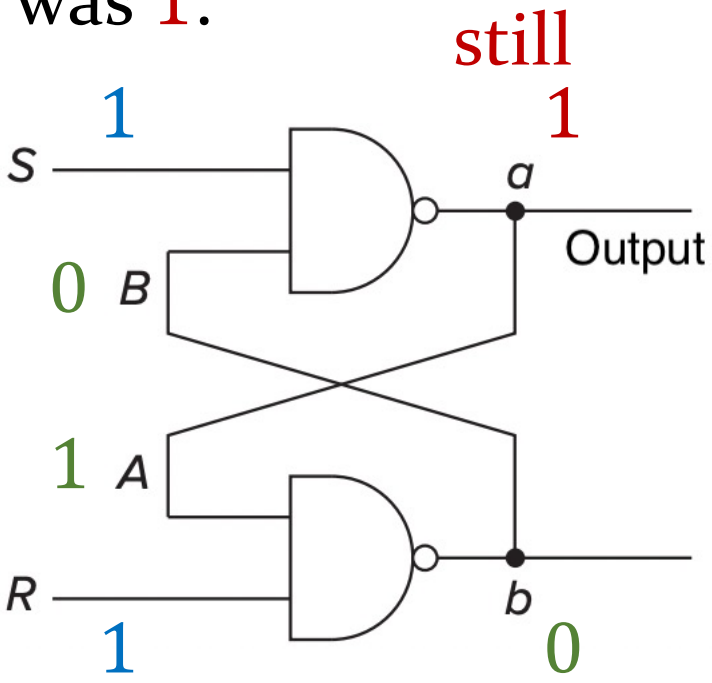


- Before we can set or reset, …

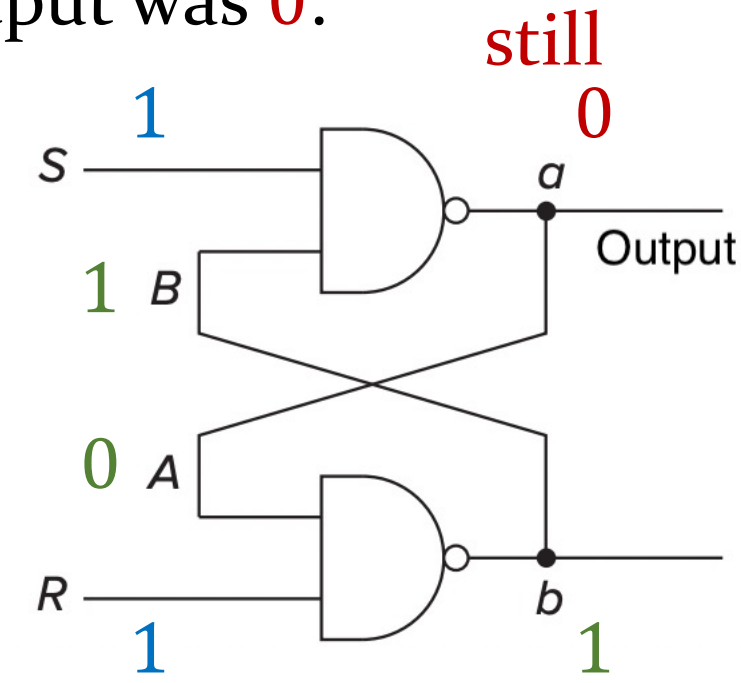
R-S Latch

- *The Quiescent (Quiet) State*: When the latch is storing a value, 0 or 1, and both input are 1. In this case, the stored value remains.

- Given output was 1:

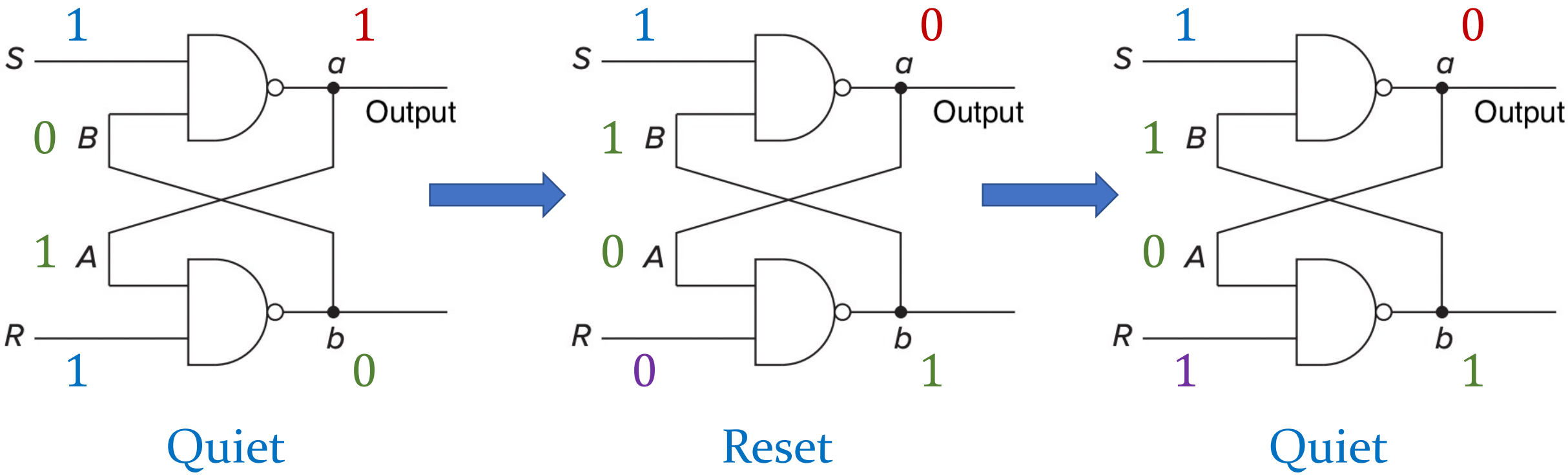


Given output was 0:



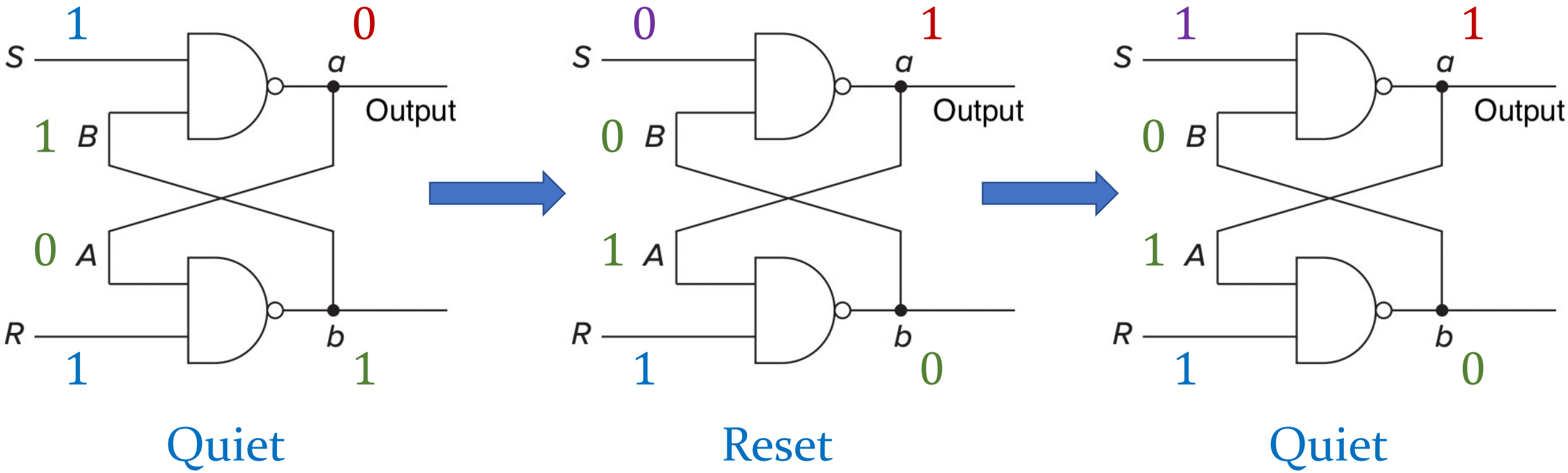
R-S Latch

- *The Reset*: In other words, was storing an 1, try to change that to a 0.
  - To reset, starting from Quiet, change *R* to 0, and back to Quiet.



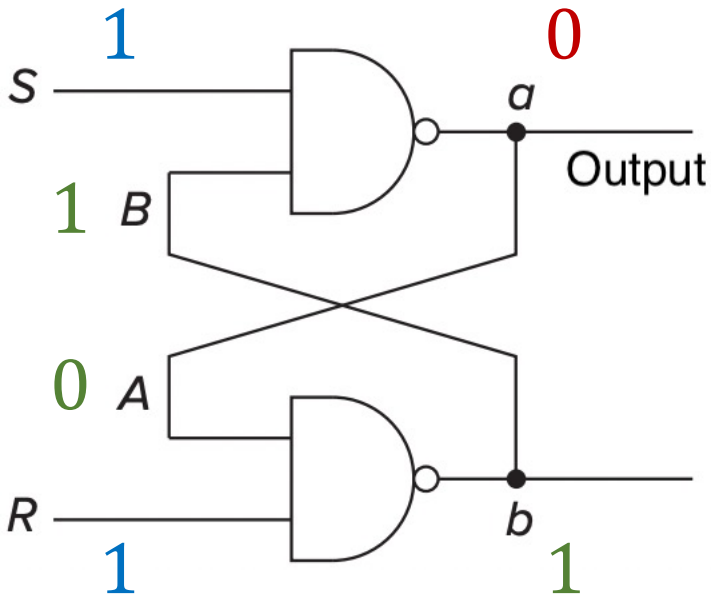
R-S Latch

- *The Set*: In other words, was storing a 0, try to change that to a 1.
  - To set, starting from Quiet, change *S* to 0, and back to Quiet.

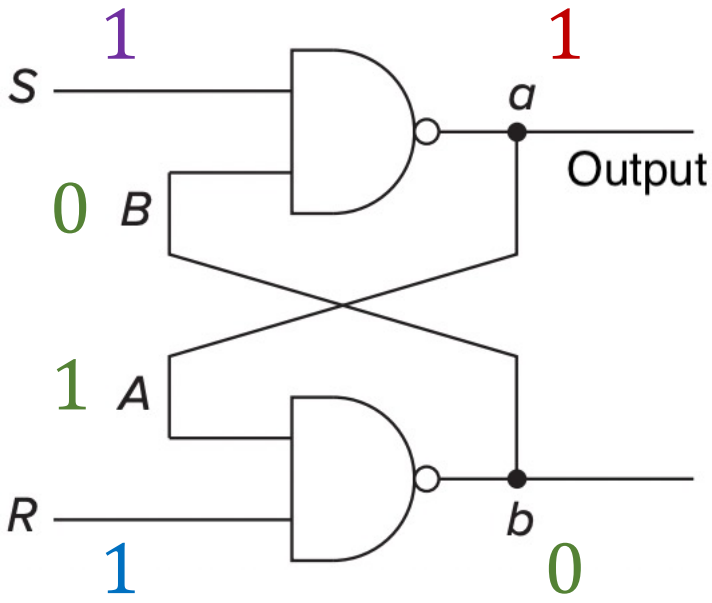


R-S Latch

- **Remarks**:
  - Latch is also referred as "Flip-Flop".
  - In a R-S latch, a, b always hold complement values.
  - Looked at $(S, R) = (1,1), (1,0), (0,1)$; $(S, R) = (0,0)$ is "not" allowed.



Quiet



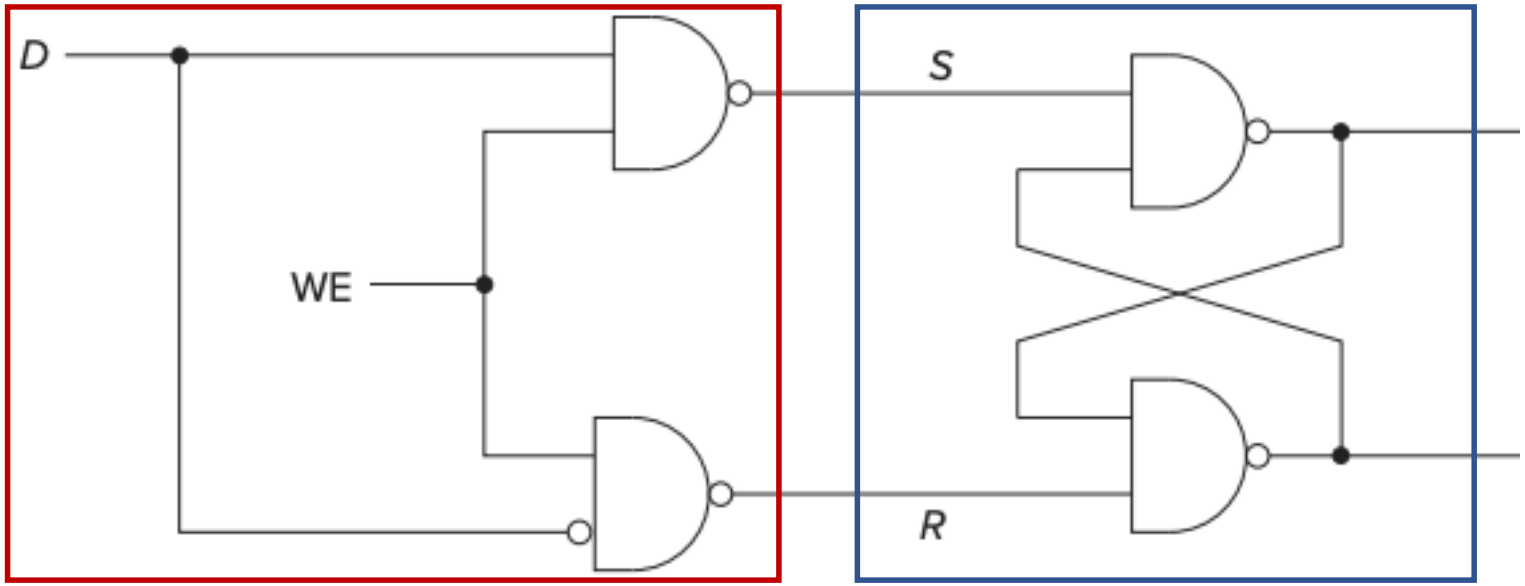
Quiet

Gated D-Latch

- To make the R-S latch practically useful, we add another control part:
  the *Gated D-Latch*.

D: data, WE: Write-Enabled

- When WE=0, not writable.
- When WE=1 (asserted),
  D write to either
  S or R, but not both.



Permission to write D to R or S

R-S Latch (blackbox)

DREW

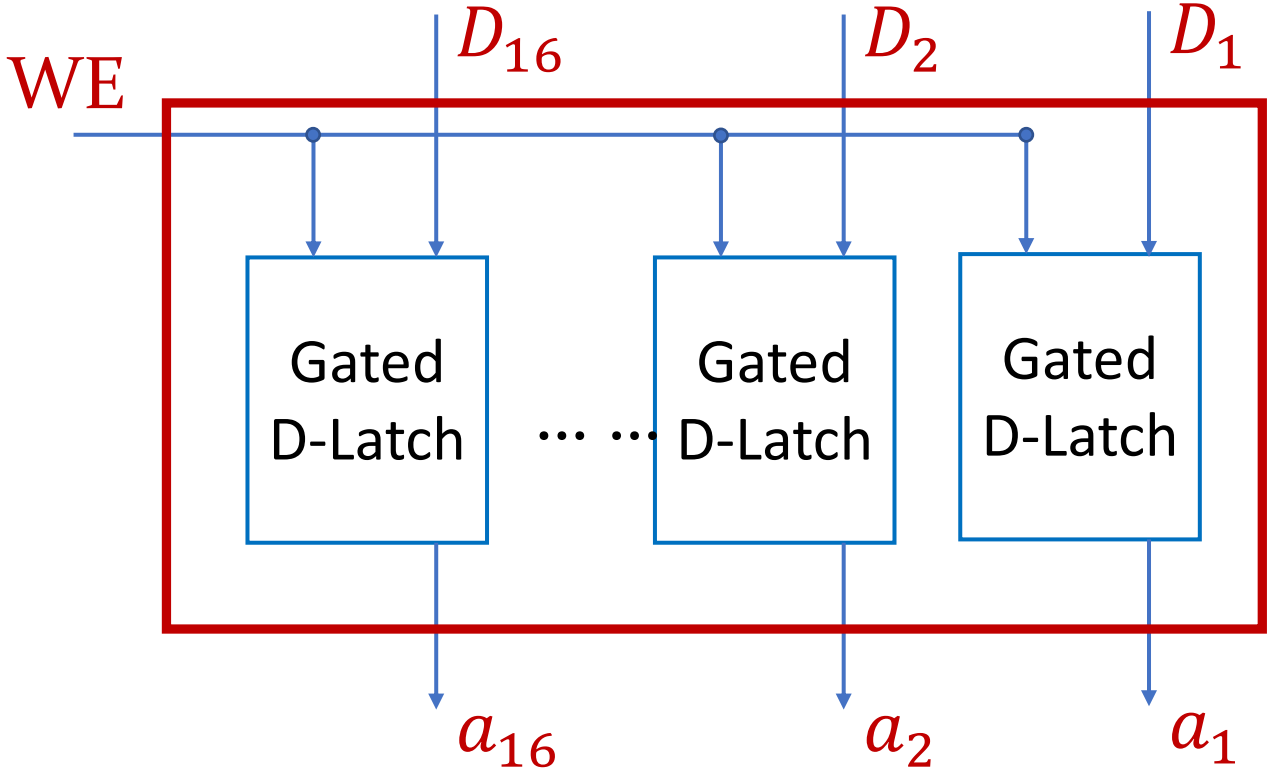
Register

- Now, it's **abstraction** time!
- To store a 1-bit value: a Gated D-Latch; how to store an n-bit value?

Combine n Gated D-Latches, where a single WE controls the write permission for all bits.

- This is known as a *Register*.

- **Ex.**: a 16-bit register A,
  $A[15:0] = E2A5$, where
  $A[3:0] = 5$.
  (or <15:0>, <3:0>, etc.)

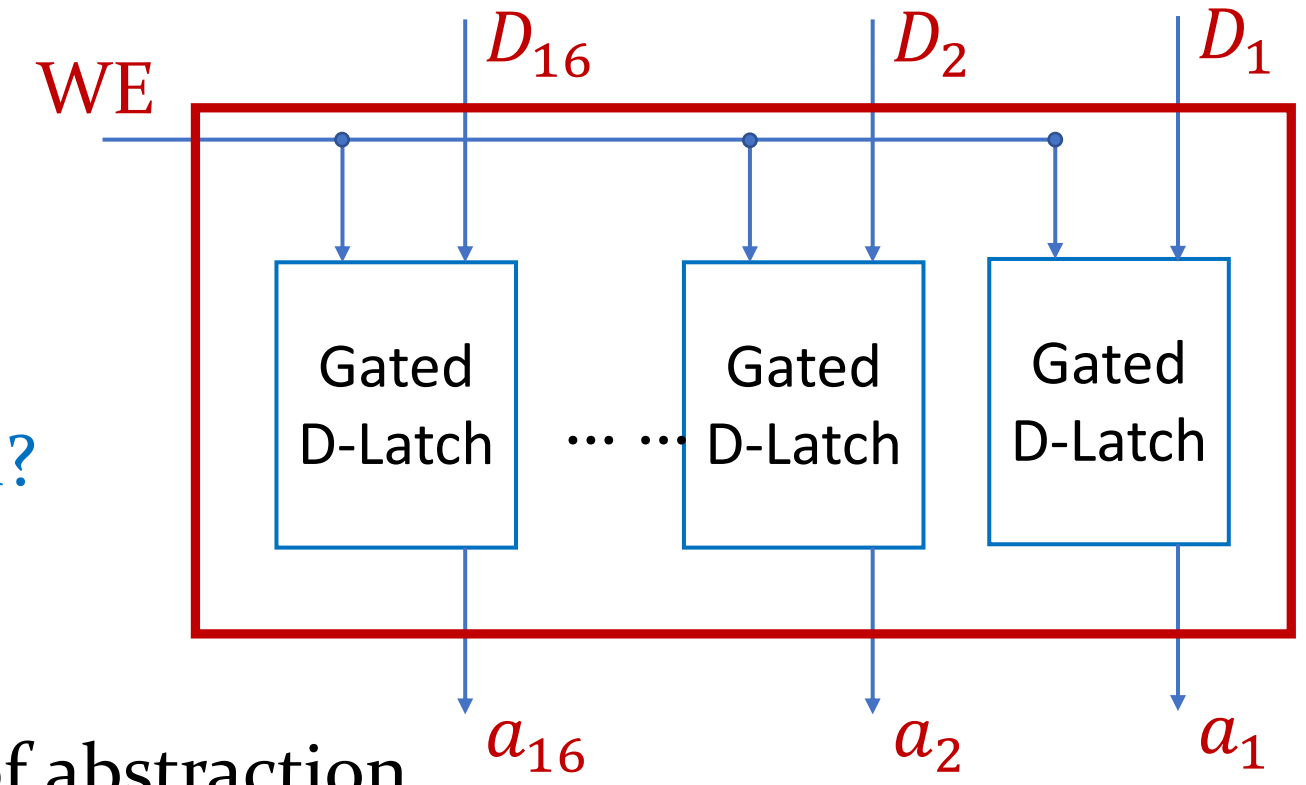


Register

- **Ex.**: a 16-bit register

Q: What do we mean by 32/64-bit processor (CPU)/operating system?

A: the width of registers in a CPU!



Turns out, on the processor-level of abstraction, register is the smallest storage unit.

Will encounter (in different forms) and modify it in the upcoming weeks.

Questions?

- **Friday:**
  - Memory
  - Sequential Logic
  - In-Class Exercise with Attendance

- **HW 2** due on 09/27 (Fri.) by 11:59pm